

第 1 课 测距

【本课目标】：

本课学习使用 openmv 通过代码测量物体与 openmv 之间的距离。

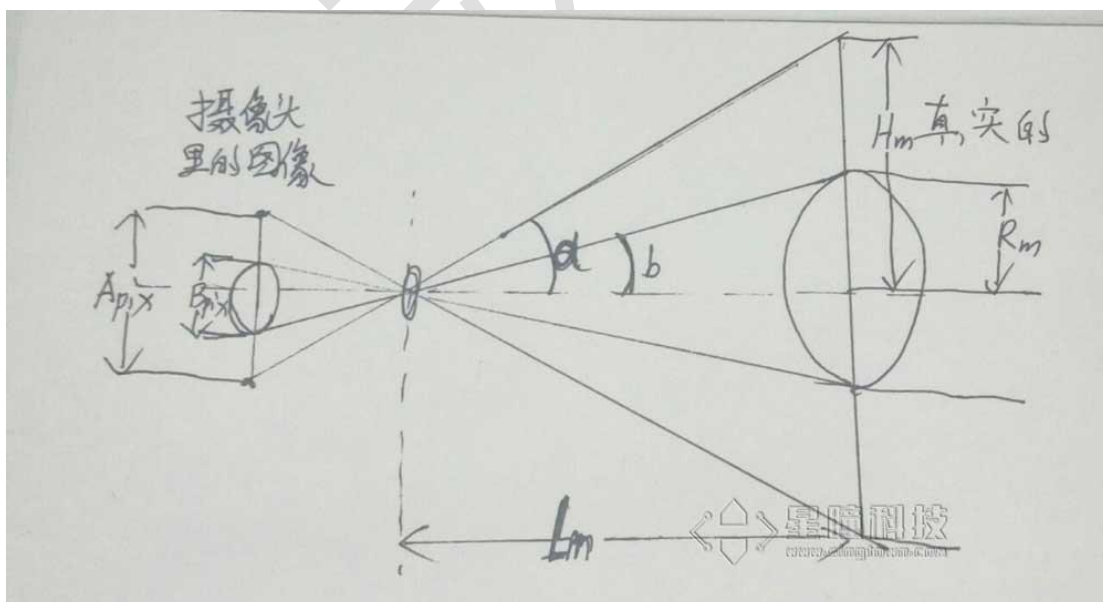
【本课内容】：

一、测距原理：

OpenMV 采用的是单目摄像头，想要实现测距，就需要选参照物，利用参照物的大小比例来计算距离。众所周知，物体距离摄像头越远，摄像头里物体的大小就越小，那么问题来了？这个关系到底是什么呢？

(注：此处的数学几何问题，仅涉及到高中数学三角函数部分，不想看的，直接看结论也可)

为了简化问题，我们看一下图：



由左边的摄像头里的几何关系可得知：

$$\tan(a) = \frac{Apix}{2L'}$$

$$\tan(b) = \frac{Bpix}{2L}$$

所以有 (1 式)

$$\frac{\tan(a)}{\tan(b)} = \frac{Apix}{Bpix}$$

由右边的真实环境里的几何关系得知：

$$\tan(b) = \frac{Rm}{Lm}$$

带入 (1 式)，可得 (结论公式)：

$$Lm * Bpix = \frac{Rm * Apix}{\tan(a)}$$

上面就是最终我们想知道的关系啦！

等号左边的 Lm 是长度， $Bpix$ 是摄像头中，球所占的像素（直径的像素）。等号右边呢， Rm 是球真实的半径， $Apix$ 是固定的像素， a 是视角的一半。

所以这个公式告诉我们的就是：

实际长度和摄像头里的像素成反比

简化就是

距离 = 一个常数/直径的像素

具体操作步骤呢，就是先测出这个常数的值，就是先让物体距离摄像头 10cm，打印出摄像头里直径的像素值，然后相乘，就得到了 k 的值！

然后 距离=这个常数/摄像头里像素点.

注意：此处是利用参照物的大小比例来计算距离，此种方法只适用于特定大小物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了。

二、代码

```

1.  # 测距
2.  # 注意：此处是利用参照物的大小比例来计算距离。
3.  # 此种方法只适用于特定大小的物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了
4.  import sensor, image, time
5.
6.  # 此处通过识别物体的颜色来测距，当然也可换成其他方式来识别
7.  target_threshold = (33, 98, 37, 94, 75, -27)
8.  #target_threshold = (10, 63, 9, 127, -20, 55)
9.
10. K = 430 # 对某个物体进行测距的第一次需要先确定比例系数 K 的大小
11. # K = 直径的像素 * 距离。
12. # 第一次时，可以将小球放在距离摄像头 10cm 处的位置，测得 Lm 的大小之后，K = 10 * Lm
13. # 比如在 10cm 处时，直径的像素为 43，则 K = 10 * 43 = 430，具体情况以你的实际情况为准
14.
15. # 初始化函数
16. def init_setup():
17.     global sensor, clock # 设置为全局变量
18.     sensor.reset() # 初始化感光元件
19.     sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
20.     sensor.set_framesize(sensor.QQVGA) # 将图像大小设置为 160x120
21.     sensor.skip_frames(10) # 跳过 10 帧，使以上设置生效
22.     sensor.set_auto_whitebal(False) # 关闭白平衡
23.     clock = time.clock() # 创建时钟对象
24.
25. # 主函数
26. def main():
27.     while 1:
28.         clock.tick() # 更新 FPS 帧率时钟
29.         img = sensor.snapshot() # 拍一张照片并返回图像
30.

```

```
31.         blobs = img.find_blobs([target_threshold]) # 找到图像中的红色区域
32.         if len(blobs) == 1:
33.             b = blobs[0]
34.             img.draw_rectangle(b[0:4]) # 将红色区域框出来
35.             img.draw_cross(b[5], b[6]) # 在红色中心位置画十字
36.             Lm = (b[2]+b[3])/2 # 获取像素点
37.             length = K/Lm # 通过比例系数 K 得到物体距离
38.             print('摄像头距离物体: %s cm' % length) # 打印距离值
39.             #print(Lm) # 打印像素点数, 在你对一个新的物体进行测距时, 需要先获取它的
            # 像素点数来得到比例系数 K
40.             #print(clock.fps()) # 打印帧率, 注: 实际 FPS 更高, 流 FB 使它更慢。
41.
42. # 程序入口
43. if __name__ == '__main__':
44.     init_setup() # 执行初始化函数
45.     try:
46.         main() # 执行主函数
47.     except:
48.         pass
```

三、代码使用方法

1. 在资料中的‘源码例程’文件夹中找到 distance_detection.py, 然后在 openmv ide 中打开。

2. 确定要测距的物体, 比如这里我要测一个红色木块与 openmv 之间的距离, 首先需要将代码中的 target_threshold = (33, 98, 37, 94, 75, -27) 处的阈值范围替换成红色木块的阈值。

阈值查找方式这里主要介绍两种, 都是在 openmv ide 中来操作的:

阈值查找方式一:

- (1) 在 openmv ide 中运行刚才打开的源码文件,
- (2) 然后将要测距的物体放在摄像头画面内, (注意不要有相近颜色的物体干扰, 否则效果不好)
- (3) 然后点击红色×号按钮停止运行程序, 此时就可以在右上方的帧缓冲区中

看到图像画面，



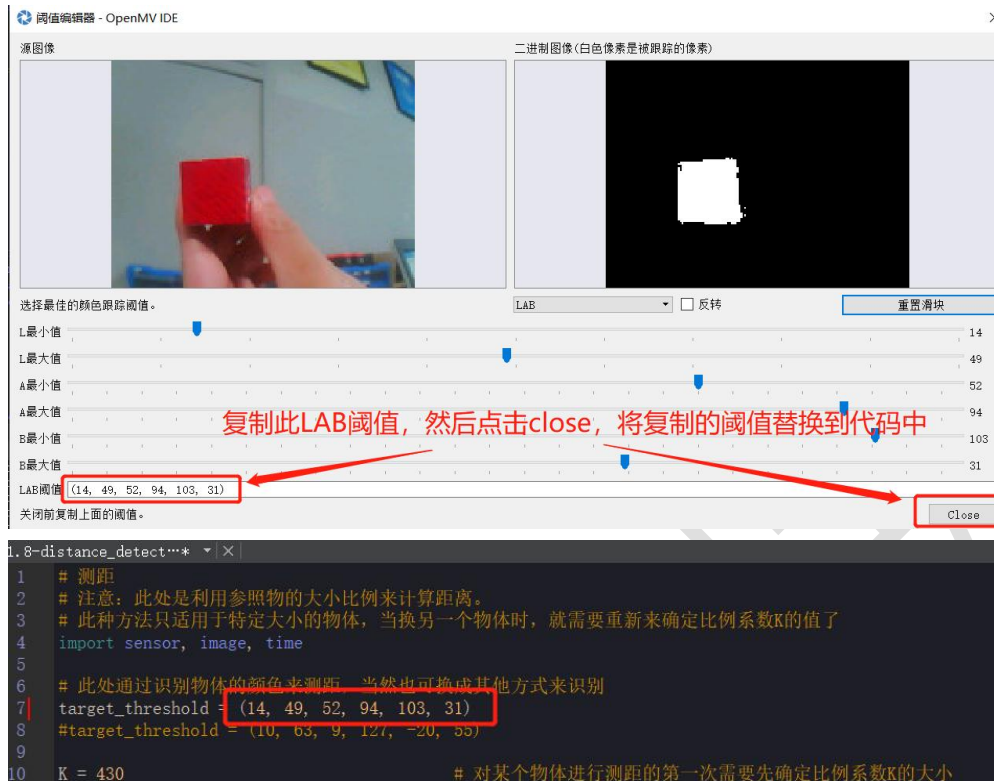
(4) 然后点击'工具'-->'机器视觉'-->'阈值编辑器'，选择'帧缓冲区'



(5) 看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。



阈值查找方式二：

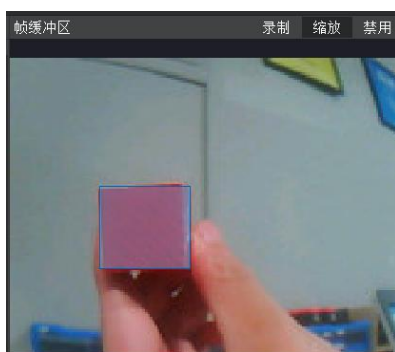
- (1) 在 openmv ide 中运行刚才打开的源码文件，
- (2) 然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）
- (3) 然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



- (4) 在'直方图'中选择'LAB 色彩空间'，



(5) 在‘帧缓冲区’中用鼠标左键将要识别的颜色框出来，比如这里我将红色木块框出来了



(6) 观察‘直方图’中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值)，如下图 L 最小 34，最大 43，A 最小 58，最大 68，B 最小 37，最大 50，则 LAB 阈值为(34,43,58,68,37,50)，替换到代码中使用就可以了。




```

1.8-distance_detect*** ▾|×|
1  # 测距
2  # 注意：此处是利用参照物的大小比例来计算距离。
3  # 此种方法只适用于特定大小的物体，当换另一个物体时，就需要重新来确定比例系数K的值了
4  import sensor, image, time
5
6  # 此处通过识别物体的颜色来测距，当然也可换成其他方式来识别
7  target_threshold = (34, 43, 58, 68, 37, 50)
8  #target_threshold = (10, 63, 9, 127, -20, 55)
9
10 K = 430 # 对某个物体进行测距的第一次需要先确定比例系数K
11 # K = 直径的像素 * 距离。
12 # 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm的大小之后，K = 10 * Lm
13 # 比如在10cm处时，直径的像素为43，则K = 10 * 43 = 430，具体情况以你的实际情况为准
14

```

3.确定并替换好代码中的颜色阈值之后，将代码中的 `print('摄像头距离物体: %s' % length)` 注释掉，将 `#print(Lm)` 取消注释，也就是将下面第一张图改为第二张图的样子：

```

25 # 主函数
26 def main():
27     while 1:
28         clock.tick() # 更新FPS帧率时钟
29         img = sensor.snapshot() # 拍一张照片并返回图像
30
31         blobs = img.find_blobs([target_threshold]) # 找到图像中的红色区域
32         if len(blobs) == 1:
33             b = blobs[0]
34             img.draw_rectangle(b[0:4]) # 将红色区域框出来
35             img.draw_cross(b[5], b[6]) # 在红色中心位置画十字
36             Lm = (b[2]+b[3])/2 # 获取像素点
37             length = K/Lm # 通过比例系数K得到物体距离
38             print('摄像头距离物体: %s cm' % length) # 打印距离值
39             #print(Lm) # 打印像素点数，在你对一个新的物体进行测距时，需
40             #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。
41

```

```

29         img = sensor.snapshot() # 拍一张照片并返回图像
30
31         blobs = img.find_blobs([target_threshold]) # 找到图像中的红色区域
32         if len(blobs) == 1:
33             b = blobs[0]
34             img.draw_rectangle(b[0:4]) # 将红色区域框出来
35             img.draw_cross(b[5], b[6]) # 在红色中心位置画十字
36             Lm = (b[2]+b[3])/2 # 获取像素点
37             length = K/Lm # 通过比例系数K得到物体距离
38             #print('摄像头距离物体: %s cm' % length) # 打印距离值
39             print(Lm) # 打印像素点数，在你对一个新的物体进行测距时，需要先获取它的像
40             #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。
41

```

4.再次运行此代码，然后将红色木块放在距离 openmv 摄像头正前方 10cm 处，观察此时‘串行端口’中的输出值，当输出值稳定后，然后停止运行程序。

```

<
串行终端
43.0
43.0
43.0
43.0
43.0

```


此时就可以计算比例系数 K 的值了，此处 $K=10*43=430$ ，（每个物体的比例系数可能都不一样，视具体情况来定）

5.将代码中的 K 赋值为 430（你上一步计算的 K 是多少，这里就替换为多少，我这里是 430）

```

8 #target_threshold = (10, 63, 9, 127, -20, 55)
9
10 K = 430 # 对某个物体进行测距
11 # K = 直径的像素 * 距离。
12 # 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm
13 # 比如在10cm处时，直径的像素为43，则K = 10 * 43 = 430，

```

6.将第 3 步还原，还原到如下图所示：

```

25 # 主函数
26 def main():
27     while 1:
28         clock.tick() # 更新FPS帧率时钟
29         img = sensor.snapshot() # 拍一张照片并返回图像
30
31         blobs = img.find_blobs([target_threshold]) # 找到图像中的红色区域
32         if len(blobs) == 1:
33             b = blobs[0]
34             img.draw_rectangle(b[0:4]) # 将红色区域框出来
35             img.draw_cross(b[5], b[6]) # 在红色中心位置画十字
36             Lm = (b[2]+b[3])/2 # 获取像素点
37             length = K/Lm # 通过比例系数K得到物体距离
38             print('摄像头距离物体: %s cm' % length) # 打印距离值
39             #print(Lm) # 打印像素点数，在你对一个新的物体进行测距时，需
40             #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。
41

```

7.然后准备工作就完成了，现在可以开始运行程序了，程序运行后，将要测距的物体在 openmv 摄像头正前方任意位置，然后观察串行端口中的输出值是否正确。

【本课小结】：

本节我们学习了使用 openmv 通过代码测量物体与 openmv 之间的距离。

注意：此处是利用参照物的大小比例来计算距离，此种方法只适用于特定大小物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了。

第 2 课 测量物体大小

【本课目标】：

本课学习通过代码测量物体实际大小。

【操作步骤】：

一、原理

利用固定距离（此处我们使用的是 10cm）和比例系数 K 来计算物体实际大小。

实际大小 = 直径的像素 * 比例系数 K

注意：当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处。

二、代码

```
1.  # 测量物体实际大小
2.  # 注意：此处是利用固定距离和比例系数来计算物体大小。
3.  # 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处
4.  import sensor, image, time
5.
6.  # 此处通过识别物体的颜色来测大小
7.  target_threshold = (13, 58, 37, 94, 75, -27)
8.
9.  K = 0.07                                # 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个
    距离处，此处我们使用的是 10cm 处
10. # 实际大小 = 直径的像素 * K。
11. # 第一次时，可以将小球放在距离摄像头 10cm 处的位置，测得 Lm 的大小之后，K = 实际大小 / Lm
12. # 比如在 10cm 处时，直径的像素为 43，小球实际大小为 3cm，则 K = 3 / 43 = 0.07，具体情况以你的实际情况为准
13.
14. # 初始化函数
15. def init_setup():
16.     global sensor, clock                # 设置为全局变量
17.     sensor.reset()                      # 初始化感光元件
18.     sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
19.     sensor.set_framesize(sensor.QQVGA)  # 将图像大小设置为 160x120
20.     sensor.skip_frames(10)              # 跳过 10 帧，使以上设置生效
```

```

21.     sensor.set_auto_whitebal(False)      # 关闭白平衡
22.     clock = time.clock()                # 创建时钟对象
23.
24.     # 主函数
25.     def main():
26.         while 1:
27.             clock.tick()                  # 更新 FPS 帧率时钟
28.             img = sensor.snapshot()       # 拍一张照片并返回图像
29.
30.             blobs = img.find_blobs([target_threshold]) # 找到图像中的目标区域
31.             if len(blobs) == 1:
32.                 b = blobs[0]
33.                 img.draw_rectangle(b[0:4]) # 将目标区域框出来
34.                 img.draw_cross(b[5], b[6]) # 在目标中心位置画十字
35.                 Lm = (b[2]+b[3])/2        # 获取像素点
36.                 w = K * b[2]              # 通过比例系数 K 和像素值得到物体实际宽度
37.                 h = K * b[3]              # 通过比例系数 K 和像素值得到物体实际高度
38.                 print('物体实际宽度为 %s cm' % w) # 打印物体实际宽度值
39.                 print('物体实际高度为 %s cm' % h) # 打印物体实际高度值
40.                 #print(Lm)                # 打印像素点数, 在你对一个新的物体进行测大小时, 需要将该物体
                 # 放在求比例系数 K 时的那个距离处, 此处我们使用的是 10cm 处
41.                 #print(clock.fps())        # 打印帧率, 注: 实际 FPS 更高, 流 FB 使它更慢。
42.
43.     # 程序入口
44.     if __name__ == '__main__':
45.         init_setup() # 执行初始化函数
46.         try:
47.             main()   # 执行主函数
48.         except:
49.             pass

```

三、代码使用方法

- 1.在资料中的‘源码例程’文件夹中找到 size_detection.py, 然后在 openmv ide 中打开。
- 2.先确定比例系数 K 的大小, 所以此时我这里找了一个红色木块, 长宽均为 3cm (你也可以找其他颜色和大小的物体), 这里同样是需要先将代码中的 target_threshold = (33, 98, 37, 94, 75, -27) 处的阈值范围替换成红色木块的阈值。

阈值查找方式这里主要介绍两种，都是在 openmv ide 中来操作的：

阈值查找方式一：

- (1) 在 openmv ide 中运行刚才打开的源码文件，
- (2) 然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）
- (3) 然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



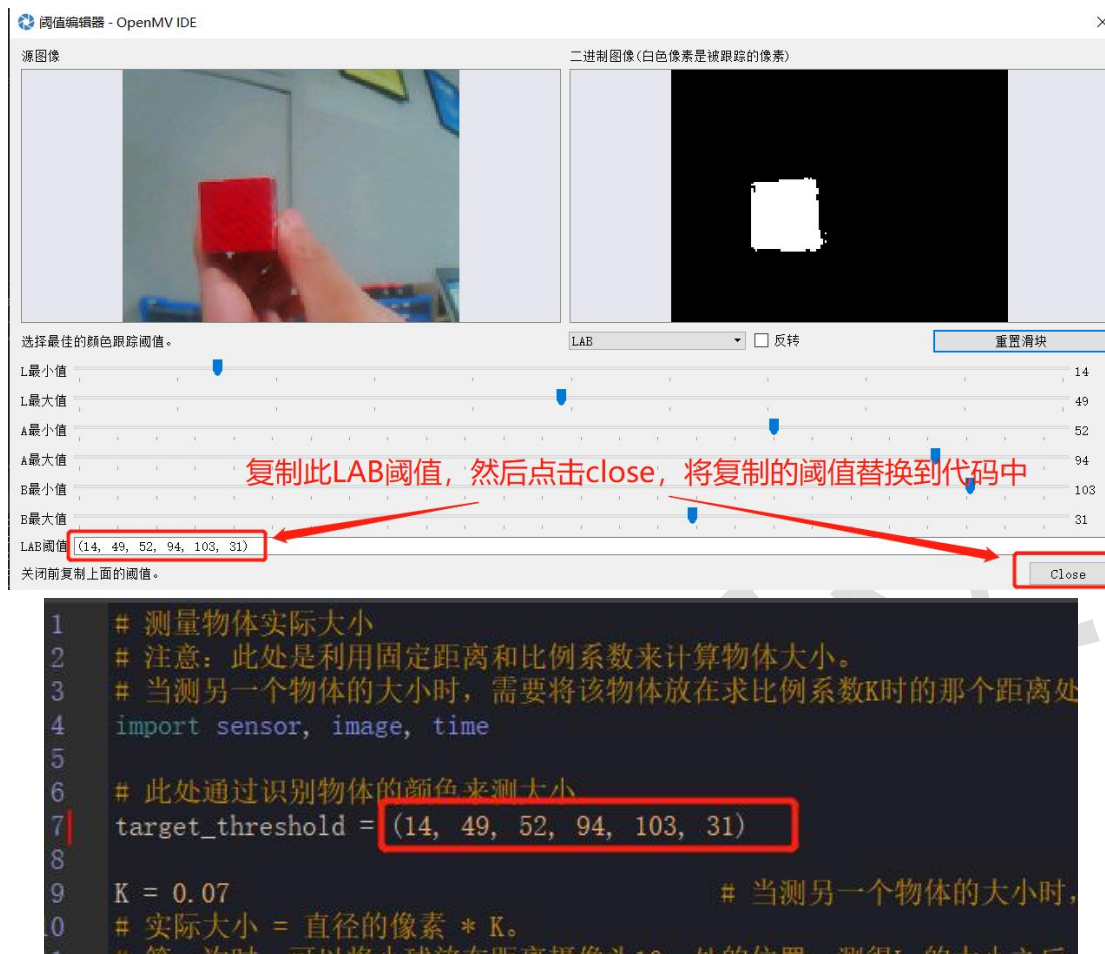
- (4) 然后点击‘工具’-->‘机器视觉’-->‘阈值编辑器’，选择‘帧缓冲区’



- (5) 看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。



阈值查找方式二：

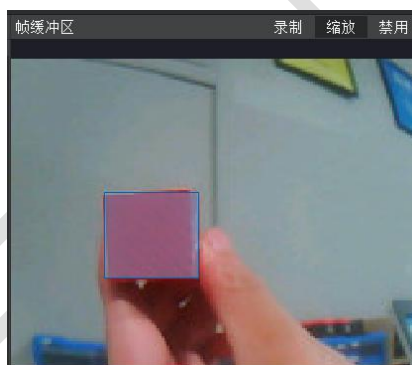
- (1) 在 openmv ide 中运行刚才打开的源码文件，
- (2) 然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）
- (3) 然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



(4) 在'直方图'中选择'LAB 色彩空间',



(5) 在'帧缓冲区'中用鼠标左键将要识别的颜色框出来, 比如这里我将红色木块框出来了



(6) 观察'直方图'中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值), 如下图 L 最小 34, 最大 43, A 最小 58, 最大 68, B 最小 37, 最大 50, 则 LAB 阈值为(34,43,58,68,37,50), 替换到代码中使用就可以了。




```

1  # 测量物体实际大小
2  # 注意：此处是利用固定距离和比例系数来计算物体大小。
3  # 当测另一个物体大小时，需要将该物体放在求比例系数K时的那个距离
4  import sensor, image, time
5
6  # 此处通过识别物体的颜色来测大小
7  target_threshold = (34, 43, 58, 68, 37, 50)
8
9  K = 0.07 # 当测另一个物体大小时
10 # 实际大小 = 直径的像素 * K。

```

3. 确定并替换好代码中的颜色阈值之后，将代码中的 `print('物体实际宽度为%s cm' % w)` 和 `print('物体实际高度为%s cm' % h)` 注释掉，将 `#print(Lm)` 取消注释，也就是将下面第一张图改成第二张图的样子：

```

24 # 主函数
25 def main():
26     while 1:
27         clock.tick() # 更新FPS帧率时钟
28         img = sensor.snapshot() # 拍一张照片并返回图像
29
30         blobs = img.find_blobs([target_threshold]) # 找到图像中的目标区域
31         if len(blobs) == 1:
32             b = blobs[0]
33             img.draw_rectangle(b[0:4]) # 将目标区域框出来
34             img.draw_cross(b[5], b[6]) # 在目标中心位置画十字
35             Lm = (b[2]+b[3])/2 # 获取像素点
36             w = K * b[2] # 通过比例系数K和像素值得到物体实际宽度
37             h = K * b[3] # 通过比例系数K和像素值得到物体实际高度
38             print('物体实际宽度为 %s cm' % w) # 打印物体实际宽度值
39             print('物体实际高度为 %s cm' % h) # 打印物体实际高度值
40             #print(Lm) # 打印像素点数，在你对一个新的物体进行测大小时，需要
41             #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。
42

```

```

32 b = blobs[0]
33 img.draw_rectangle(b[0:4]) # 将目标区域框出来
34 img.draw_cross(b[5], b[6]) # 在目标中心位置画十字
35 Lm = (b[2]+b[3])/2 # 获取像素点
36 w = K * b[2] # 通过比例系数K和像素值得到物体实际宽度
37 h = K * b[3] # 通过比例系数K和像素值得到物体实际高度
38 #print('物体实际宽度为 %s cm' % w) # 打印物体实际宽度值
39 #print('物体实际高度为 %s cm' % h) # 打印物体实际高度值
40 print(Lm) # 打印像素点数，在你对一个新的物体进行测大小时，需要
41 #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。
42

```

4. 再次运行此代码，然后将红色木块放在距离 openmv 摄像头正前方 10cm 处，观察此时‘串行端口’中的输出值，当输出值稳定后，然后停止运行程序。

```

< 串行终端
43.0
43.0
43.0
43.0
43.0

```

此时就可以计算比例系数 K 的值了，此处 $K=3/43=0.07$ ，（以你测得的实际为准）

5.将代码中的 K 赋值为 0.07（你上一步计算的 K 是多少，这里就替换为多少，我这里是 0.07）

```

3  # 当测另一个物体的大小时，需要将该物体放在求比例系数K时的那个距离处，此处我
4  import sensor, image, time
5
6  # 此处通过识别物体的颜色来测大小
7  target_threshold = (14, 49, 52, 94, 103, 31)
8
9  K = 0.07 # 当测另一个物体的大小时，需要将该
10 # 实际大小 = 直径的像素 * K。
11 # 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm的大小之后，K = 实际

```

6.将第 3 步还原，还原到如下图所示：

```

24 # 主函数
25 def main():
26     while 1:
27         clock.tick() # 更新FPS帧率时钟
28         img = sensor.snapshot() # 拍一张照片并返回图像
29
30         blobs = img.find_blobs([target_threshold]) # 找到图像中的目标区域
31         if len(blobs) == 1:
32             b = blobs[0]
33             img.draw_rectangle(b[0:4]) # 将目标区域框出来
34             img.draw_cross(b[5], b[6]) # 在目标中心位置画十字
35             Lm = (b[2]+b[3])/2 # 获取像素点
36             w = K * b[2] # 通过比例系数K和像素值得到物体实际宽度
37             h = K * b[3] # 通过比例系数K和像素值得到物体实际高度
38             print('物体实际宽度为 %s cm' % w) # 打印物体实际宽度值
39             print('物体实际高度为 %s cm' % h) # 打印物体实际高度值
40             #print(Lm) # 打印像素点数，在你对一个新的物体进行测大小
41             #print(clock.fps()) # 打印帧率，注：实际FPS更高，流FB使它更慢。

```

7.然后准备工作就完成了，现在可以开始运行程序了，程序运行后，将要测大小的物体（注意：如果要测的物体和确定比例系数时的颜色不一样，需要重新把被测物体的颜色阈值替换到代码中）放在 openmv 摄像头正前方 10cm 处，然后观察串行端口中的输出值是否正确。

【本课小结】：

本节我们学习了使用 openmv 通过代码测量物体的实际大小。

注意：

1. 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处。
2. 当被测物体颜色不同于确定比例系数 K 时的那个颜色时，需要在代码中将被测物体的颜色阈值替换上去。